Git Handbook

Basics and terminology

Git tracks changes locally. GitHub hosts repositories online and provides collaboration tools.

- **Working directory:** The files you are editing.
- **Staging area:** The changes selected for your next commit.
- **Commit:** A saved snapshot with a message and a unique ID.
- **Branch:** A name pointing to a commit; it advances when you commit on it.
- **HEAD:** Usually points to your current branch and therefore its latest commit.
- **Remote:** A connection to another repository, commonly named `origin`.
- **`main` vs. `origin/main`:** Your local branch versus your local record of the remote branch, updated by fetches and successful pushes.

Important commands

Command	Purpose
`git init -b main`	Start a repository in the current folder with a branch named `main`.
`git clone <url>`	Download an existing repository into a new folder.
`git remote add origin <url>`	Connect your local repository to a remote named `origin`.
`git status`	Show the current branch and staged, unstaged, or untracked files.
`git diff`	Show unstaged changes to tracked files.
`git diff --staged`	Show changes selected for the next commit.
`git add <file>`	Stage the file's current changes. Later edits need another `add`.
`git commit -m "message"`	Save staged changes locally.
`git log --oneline`	Show compact commit history.
`git branch`	List local branches.
`git switch -c feature`	Create and switch to a branch named `feature`. You can choose different branch names.
`git switch main`	Switch to the existing `main` branch.
`git merge feature`	Integrate the branch named `feature` into your current branch.
`git rebase main`	Replay your current branch's commits on top of `main`.
`git fetch origin`	Download updates without integrating them into your current branch.

Command	Purpose
`git pull`	Fetch and integrate remote updates; flags and configuration control how.
`git push`	Upload local commits and update the corresponding remote branch.
`git push -u origin feature`	Push the branch and set its upstream for later pulls and pushes.
`git stash` / `git stash pop`	Shelve uncommitted tracked changes / reapply the latest stash and remove it if successful.
`git restore --staged <file>`	Unstage changes while keeping your edits.
`git restore <file>`	Replace working-file content with its staged version, discarding unstaged edits.
`git revert <commit>`	Create a new commit that undoes an earlier commit.
`git reset --soft <commit>`	Move the current branch back while keeping later changes staged.

Important differences

Comparison	Difference
Commit vs. push	Commit saves locally. Push shares commits with a remote.
Init vs. clone	Init starts local history. Clone copies existing history and configures a remote.
Merge vs. rebase	Merge preserves existing commits and may add a merge commit. Rebase replays commits with new IDs, producing a linear sequence on the new base.
Fast-forward vs. merge commit	A fast-forward only advances a branch pointer. A merge commit joins histories and has multiple parents.
Fetch vs. pull	Fetch updates remote-tracking information. Pull also integrates updates into your current branch.
Revert vs. reset	Revert records an undo in history. Reset moves the branch pointer; its mode controls what happens to staged changes and files.
Pull vs. pull request	Pull updates a local branch. A pull request proposes integrating one branch into another on GitHub.

Use explicit pull behavior:

- `git pull --ff-only`: Update only when your branch can advance without combining divergent histories.
- `git pull --rebase`: Replay your local-only commits on top of the fetched branch.
- `git pull --no-rebase`: Use a merge to integrate the fetched branch.

Avoid rebasing or resetting published commits that other people are using. `reset --soft` keeps changes staged; the default `--mixed` keeps changes unstaged; `--hard` also overwrites tracked working files and can remove obstructing untracked files.

A **pull request (PR)** is a GitHub review workflow, not a Git command. Opening one does not merge anything. A PR can contain discussion, automated checks, and several commits. GitHub can integrate it using a merge commit, squash merge, or rebase merge, depending on repository settings.

A **conflict** occurs when Git cannot reconcile changes automatically. Edit the affected files into the desired final state, remove conflict markers, stage the resolution, and finish the merge or rebase.

References: merge, rebase, pull, reset, and GitHub pull requests.

Try it yourself

You can download Git from here https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

Work through these exercises **in order**, one code block at a time. Use Bash on Linux/macOS or Git Bash on Windows, with Git 2.28 or newer. All project content will be plain text files.

Run commands in the same terminal. Unless instructed otherwise, stay in `git-playground`. If a command fails unexpectedly, fix it before continuing. Expected failures are labeled. If Git opens a pager, press `q` to exit.

Shell reminders: `echo` writes text, `>` creates or replaces a file, `>>` appends, `cat` displays a file, and `ls` lists files. These are shell commands, not Git commands.

1. Create an empty GitHub repository

1. Sign in to GitHub and create a repository named `git-playground`. Here, "project" means a repository, not a GitHub Projects board.
2. Choose private or public visibility.
3. Leave README, `.gitignore`, and license initialization disabled so the repository is empty.
4. Copy its HTTPS URL: `https://github.com/YOUR_USERNAME/git-playground.git`.

Replace `YOUR_USERNAME` below with your actual username. For HTTPS pushes, use your configured credential manager or a personal access token when prompted for a password; your GitHub account password will not work. See GitHub authentication.

2. Create a folder and connect it to GitHub

Start in a parent directory where you want to keep this exercise:

```
mkdir git-playground
cd git-playground
git init -b main

git config user.name "Your Name"
git config user.email "YOUR_EMAIL"

# these commands creates files and places content inside.
```

```
echo "My Git practice notes" > notes.txt
echo "Hello world" > greeting.txt
echo "color=blue" > settings.txt

# check the current working tree
git status
# stage your files for commit
git add notes.txt greeting.txt settings.txt
git diff --staged
# commit your changes
git commit -m "Create practice files"

# connect your local git project with your online repository
git remote add origin https://github.com/YOUR_USERNAME/git-playground.git
git remote -v
# push your local commit to the online repository
git push -u origin main
```

Replace the name and email with your own details; GitHub's provided no-reply email also works. These settings apply only to this repository and identify commits; they do not authenticate you.

Check: Refresh GitHub. All three text files should appear on `main`.

Further documentation: Adding local code to GitHub.

3. Inspect, stage, commit, and push an edit

```
echo "I can inspect and save changes." >> notes.txt
git status
git diff
git add notes.txt
git diff
git diff --staged
git commit -m "Add a practice note"
git log --oneline -3
git push
```

Check: Before staging, `git diff` shows your edit. After staging, it is empty and `git diff --staged` shows the edit. After committing, your working tree is clean. Only the push updates GitHub.

4. Create a branch and merge it

```
git switch -c feature-list
echo "Practice branching" > tasks.txt
git add tasks.txt
git commit -m "Add practice tasks"
git branch

git switch main
```

```
ls
git merge --no-ff feature-list -m "Merge practice tasks"
cat tasks.txt
git log --graph --oneline --all -8
```

Check: `tasks.txt` disappears when you switch to `main` and returns after the merge. `--no-ff` deliberately creates a merge commit, even though this example could otherwise fast-forward.

5. Trigger and resolve a merge conflict

Change the same original line differently on two branches:

```
git switch -c friendly-greeting
echo "Hello friends" > greeting.txt
git add greeting.txt
git commit -m "Use a friendly greeting"

git switch main
echo "Hello team" > greeting.txt
git add greeting.txt
git commit -m "Use a team greeting"

git merge friendly-greeting
```

Expected failure: Git reports a content conflict. Inspect it:

```
git status
cat greeting.txt
```

With the default conflict style, the file contains:

```
<<<<<<< HEAD
Hello team
=======
Hello friends
>>>>>>> friendly-greeting
```

The top block is the current branch's version; the bottom is the incoming branch's version. Other conflict styles can also show the original line.

First practice canceling the merge, then trigger it again:

```
git merge --abort
cat greeting.txt
```

```
git merge friendly-greeting
```

Expected failure again: The conflict returns. Resolve it by choosing a final sentence. Overwriting this one-line file also removes all conflict markers:

```
echo "Hello team and friends" > greeting.txt
git add greeting.txt
git commit -m "Resolve greeting conflict"
git status
```

Check: The file contains your combined sentence, the working tree is clean, and the merge is complete.

6. Trigger and resolve a rebase conflict

Create a branch that changes the color, then change the same line on `main`:

```
git switch -c color-update
echo "color=green" > settings.txt
git add settings.txt
git commit -m "Choose green"
git log --oneline -1

git switch main
echo "color=red" > settings.txt
git add settings.txt
git commit -m "Choose red"

git switch color-update
git rebase main
```

Expected failure: Git cannot replay "Choose green" over "Choose red" automatically. Unlike merging, this operation rewrites commits on `color-update`; it does not move `main`.

Inspect, cancel, and retry:

```
cat settings.txt
git rebase --abort
cat settings.txt
git rebase main
```

Expected failure again: After aborting, the file was green; retrying produces the conflict again. Resolve it with a third color:

```
echo "color=purple" > settings.txt
git add settings.txt
```

```
git -c core.editor=true rebase --continue
git log --oneline -3
```

`-c core.editor=true` accepts the existing commit message without opening an editor for this command. During rebase, the `HEAD` side of a conflict represents the new base plus commits already replayed; the other side is the commit being replayed.

Check: "Choose green" now has a different commit ID and follows "Choose red". Its message remains unchanged even though you resolved the content to purple.

Bring the result into `main`:

```
git switch main
git merge --ff-only color-update
cat settings.txt
git push
```

Check: The result is `color=purple`. This merge advances `main` without adding a merge commit.

7. Open and merge a pull request

```
git switch -c pr-notes
echo "This file arrived through a pull request." > review.txt
git add review.txt
git commit -m "Add review notes"
git push -u origin pr-notes
```

On GitHub:

1. Open the repository's **Pull requests** tab and select **New pull request**.
2. Set **base** to `main` and **compare** to `pr-notes`.
3. Review the changes, enter a title, and create the PR.
4. Open **Files changed** to inspect the proposed addition.
5. Choose **Create a merge commit** from the merge options, then merge and confirm. This keeps the branch's original commit in `main` for this exercise.

Check: GitHub's `main` contains `review.txt`. Your local `main` has not updated yet.

8. See the difference between fetch and pull

```
git switch main
git fetch origin
git log --oneline main..origin/main
git diff main origin/main
ls
```

Check: Git knows about the remote commits, but `review.txt` is still absent from your working folder. The `main..origin/main` range means commits reachable from `origin/main` but not from `main`.

```
git pull --ff-only
cat review.txt
git branch -d pr-notes
```

Check: The file now appears. Deleting the merged local branch leaves its commits in history; this command does not delete the GitHub branch.

9. Clone a second copy and simulate a teammate

Create a sibling folder containing a separate clone:

```
cd ..
git clone https://github.com/YOUR_USERNAME/git-playground.git git-
playground-peer
cd git-playground-peer
git config user.name "Your Name"
git config user.email "YOUR_EMAIL"

echo "A teammate added this note." > team.txt
git add team.txt
git commit -m "Add teammate note"
git push
```

Return to your first copy and commit another change without pulling:

```
cd ../git-playground
echo "My simultaneous local change." >> notes.txt
git add notes.txt
git commit -m "Add local work"
git push
```

Expected failure: The push is rejected because GitHub has commits your local branch lacks. This is a history divergence, not yet a file-content conflict.

```
git fetch origin
git log --oneline --left-right main...origin/main
git pull --ff-only
```

Expected failure: A fast-forward cannot reconcile the divergent branches. In the log, `<` marks local-only commits and `>` marks remote-only commits.

Rebase the unpublished local work onto the remote work:

```
git pull --rebase
git log --oneline -4
cat team.txt
git push
```

Check: Both changes survive. No content conflict occurs because the commits changed different files.

10. Compare pulling with a merge

Create another remote change from the second copy:

```
cd ../git-playground-peer
git pull --ff-only
echo "Another teammate change." >> team.txt
git add team.txt
git commit -m "Extend teammate note"
git push

cd ../git-playground
echo "Another local change." >> notes.txt
git add notes.txt
git commit -m "Extend local notes"
git pull --no-rebase --no-edit
git log --graph --oneline -6
git push
```

Check: This time Git adds a merge commit connecting the two histories. The existing local commit keeps its ID. `--no-edit` accepts Git's generated merge message.

11. Stash unfinished work

```
echo "An unfinished idea." >> notes.txt
git diff
git stash push -m "Unfinished notes"
git status
git stash list
cat notes.txt

git switch feature-list
git switch main
git stash pop
git diff
git add notes.txt
git commit -m "Finish the stashed idea"
```

Check: The idea disappears while stashed and returns after `pop`. Plain stash excludes untracked files; use `git stash push -u` to include them. A pop can conflict if the destination changed; in that case the stash is retained. See stash.

12. Undo a published commit with revert

```
echo "This file was a mistake." > mistake.txt
git add mistake.txt
git commit -m "Add an unwanted file"
git push

git revert --no-edit HEAD
git log --oneline -3
ls
git push
```

Check: `mistake.txt` disappears. The original commit remains in history, followed by a new commit reversing it. `HEAD` selected the latest commit at the time of the revert.

13. Redo an unpublished commit with reset

Use a fresh local branch:

```
git switch -c reset-practice
echo "Draft wording" > draft.txt
git add draft.txt
git commit -m "Temporary draft"

git reset --soft HEAD~1
git status
git diff --staged
git log --oneline -2
```

Check: The temporary commit is no longer at the branch tip, but `draft.txt` still exists and its addition is staged. `HEAD~1` means the first parent of the current commit.

Improve the draft and commit again:

```
echo "Final wording" > draft.txt
git add draft.txt
git commit -m "Add the final draft"
git switch main
git merge --ff-only reset-practice
```

Check: The new commit contains the final wording. The temporary commit was never pushed.

14. Unstage an edit, then discard it

```
echo "An edit I do not want." >> notes.txt
git add notes.txt
git restore --staged notes.txt
git status
git diff

git restore notes.txt
git status
```

Check: Unstaging keeps the edit in your file. Restoring afterward discards it because the staged version now matches the last commit. Discard only the practice edit here; Git generally cannot recover edits that were never committed or stashed.

15. Finish and inspect the result

```
git push
git status
git log --graph --oneline --decorate --all -25
```

Check: You are on `main`, your working tree is clean, and GitHub has the final files. The history includes branch commits, merge commits, replayed commits, and an explicit revert.